Cancelamento, absorção e ULP em ponto flutuante

um estudo da expressão $(x^n + c) - x^n$ em `binary32` e `binary64`

Lucas Martins Rocha

Rascunho de 4 de novembro de 2026

Nota ao leitor

Este texto está em desenvolvimento. Ele parte de um artigo curto, já escrito, sobre o comportamento da função

$$f(x) = \left(x^{10} + c\right) - x^{10}$$

em aritmética de ponto flutuante quando c é inteiro ímpar ou fracionário, e o amplia em um texto científico com fundamentos, demonstrações, variações e aplicações didáticas. O escopo é restrito aos formatos `binary32` (`float32`) e `binary64` (`float64`) do padrão IEEE 754.

Duas convenções ajudam a ler o rascunho. Caixas laranja marcam trechos *a completar*, com o escopo do que deve entrar. Caixas azuis indicam o *estado* de um resultado: o que está demonstrado neste texto, o que só foi verificado experimentalmente e o que ainda é conjectura.

Onde está o artigo. O artigo original corresponde aos Capítulos 4 (Teoria), 6 (Inteiros ímpares), 7 (Frações) e 8 (Precisões), lidos nessa ordem. Os demais capítulos o situam (Capítulos 1 a 3), generalizam (Capítulos 5 e 9) ou aplicam (Capítulos 10 e 11).

Sumário

1 Introdução **1**

1.1 Motivação . . . 1
1.2 Questão de pesquisa . . . 1
1.3 Contribuições . . . 1
1.4 Relação com a literatura . . . 2
1.4.1 O que já é conhecido . . . 2
1.4.2 O que não encontrei . . . 2
1.5 Escopo e limitações . . . 2
1.6 Organização . . . 2
1.7 Reprodutibilidade e origem do código . . . 3

2 Fundamentos de ponto flutuante **4**

2.1 Formatos binários . . . 4
2.2 Binades e ULP . . . 4
2.3 Arredondamento ao mais próximo, empate para par . . . 4
2.4 Subtração exata: o lema de Sterbenz . . . 5
2.5 Exemplos numéricos . . . 5

3 O problema e a metodologia **6**

3.1 A função . . . 6
3.2 A observação inicial . . . 6
3.3 Hipóteses . . . 6
3.4 Como $y = x^n$ é calculado . . . 6
3.5 Metodologia experimental . . . 7

4 Teoria: o modelo da ULP **8**

4.1 Notação . . . 8
4.2 A proposição principal . . . 8
4.3 Limiares e largura da zona ruidosa . . . 9
4.4 Erro em ULPs . . . 10
4.5 Quantos valores f assume em uma binade . . . 10
4.6 Efeito de borda de binade . . . 10
4.7 A região $y < c$ . . . 11
4.8 Generalização a n e p . . . 11
4.9 O que está e o que não está demonstrado . . . 11

5 Caso base: c potência de dois **12**

6 Inteiros ímpares **13**
6.1 O limiar universal . . . 13
6.2 A binade de empate . . . 13
6.3 Os degraus . . . 13
6.4 O mapa (x, c) . . . 13

7 Frações **16**
7.1 Frações diádicas . . . 16
7.2 Frações não diádicas . . . 16
7.2.1 O que significa $c = 0{,}1$ . . . 16
7.2.2 Zona larga . . . 16
7.2.3 Crescimento do erro . . . 16

8 `binary32` contra `binary64` **18**
8.1 Validação: o dente de serra . . . 19

9 Variações **20**
9.1 Outros expoentes n . . . 20
9.2 c negativo . . . 20
9.3 c da mesma ordem de y . . . 21
9.4 Como y é calculado . . . 21
9.5 FMA e precisão estendida . . . 21

10 Remédios **22**
10.1 Reescrever a expressão . . . 22
10.2 O resultado é o z do Fast2Sum . . . 22
10.3 Soma compensada . . . 22

11 Aplicações didáticas **23**
11.1 Roteiro de oficina . . . 23
11.2 Exercícios com gabarito . . . 23
11.3 Notebook interativo . . . 24
11.4 Erros comuns . . . 24

12 Conclusões e perspectivas **25**
12.1 O que se estabeleceu . . . 25
12.2 O que fica em aberto . . . 25
12.3 Perspectivas . . . 25

A Código **26**
A.1 `ulp_core.py`: modelo e experimento . . . 26
A.2 `validate.py`: validação da fórmula . . . 27
A.3 `figs.py`: figuras . . . 28

B Tabelas completas **32**

C Notação **33**

D Registro da revisão da literatura **34**
D.1 Fontes lidas . . . 34
D.2 Fontes que não consegui ler . . . 34
D.3 Busca ainda por fazer . . . 35

Lista de Figuras

6.1 $f(x)$ em `binary64` para $c = 3, 5, 101$ e 173. Faixa laranja: binade de empate ($u = 2$). Linhas pontilhadas: fronteiras de binade $x = 2^{E/10}$. O traço tracejado é o valor exato c. 15
6.2 Erro relativo $|f - c|/c$ para os inteiros ímpares de 1 a 255 (linhas) e x de 36 a 82 (colunas). Branco: exato. Preto: $f = 0$. Linha verde: $x_2(c)$ previsto. . . . 15

7.1 Erro relativo em frações diádicas (em cima, escala linear) e não diádicas (em-baixo, $|f - c|/c$ em escala logarítmica; vermelho: $f < c$). Faixa laranja: zona ruidosa $[x_1, x_2]$ prevista. 17

8.1 Zona ruidosa prevista $[x_1, x_2]$ (barras) e limites medidos (traços pretos) para valores de c de cada classe, em `binary64` e `binary32`. L = número de binades da zona. 18
8.2 Erro em ULPs, $\varepsilon = (f - c)/\mathrm{ulp}(x^{10})$, contra a parte fracionária de $q = c/\mathrm{ulp}(x^{10})$, em `binary64`. A linha preta é a previsão $\varepsilon = \mathrm{arred}(q) - q$. 19

Lista de Tabelas

2.1 Formatos IEEE 754 tratados neste texto. 4

6.1 Valores de f por binade para $c = 101$ (`binary64`). 14
6.2 Limites da zona ruidosa para c ímpar (`binary64`). 14

7.1 Erro relativo $(f - c)/c$ medido em `binary64`. 17

8.1 Zona ruidosa prevista $[x_1, x_2)$ e L para $n = 10$. 18

9.1 Limiar $x_1 = 2^{p/n}$ para c ímpar. 20

Capítulo 1

Introdução

1.1 Motivação

Matematicamente, $x^n + c - x^n$ é a função constante c. No computador, ela vira uma escada. Para x pequeno o resultado é exatamente c; a partir de um certo x, ele passa a oscilar entre valores vizinhos de c; mais adiante, assume valores sistematicamente errados e, por fim, vale 0. A causa é a distância entre números consecutivos em ponto flutuante, a *ULP*, que cresce junto com x^n.

O exemplo é pequeno, determinístico e fácil de visualizar, o que o torna um bom laboratório para três conceitos centrais da aritmética de máquina: o arredondamento ao mais próximo com desempate para par, o cancelamento e a absorção.

1.2 Questão de pesquisa

> *Como o valor de c, e não apenas o tamanho de x^n, determina onde o erro de $(x^n + c) - x^n$ começa, qual é seu tamanho e onde o resultado colapsa para zero?*

O caso em que c é potência de dois já foi explorado. Este texto trata de c inteiro ímpar, de c fracionário diádico e de c fracionário não diádico, e coloca os três casos sob uma mesma fórmula.

1.3 Contribuições

Depois da revisão da literatura da Seção 1.4, as contribuições são descritas com cautela: o mecanismo é clássico e a parte própria é a tradução explícita para (n, c, x).

1. Uma fórmula fechada para $f(x)$ em cada binade de x^n, com desempate explícito (Capítulo 4). Ela é uma consequência elementar dos resultados sobre o Fast2Sum, escrita aqui em função de x.
2. Limites x_1 e x_2 da zona ruidosa em função de $\nu(c)$ e $\lfloor \log_2 c \rfloor$, e a largura L da zona como número de bits significativos de c. Esta é a parte que não encontrei enunciada nas fontes consultadas (Seção 1.4).
3. Demonstrações elementares dos resultados acima, a partir do lema de Sterbenz e da definição de arredondamento ao mais próximo.
4. Validação experimental em `binary64` e `binary32` com 1 611 valores de c por precisão, e a medida do que a teoria não cobre (efeito de borda de binade, região $y < c$).
5. A conexão explícita com o Fast2Sum: $f = \mathrm{fl}(\mathrm{fl}(y + c) - y)$ é o valor intermediário z do algoritmo e $c - f$ é o seu erro (Capítulo 10).

1.4 Relação com a literatura

1.4.1 O que já é conhecido

Com $a = y$ e $b = c$, a expressão $f(x)$ é exatamente o valor $z = \mathrm{fl}(s-a)$ do algoritmo Fast2Sum, onde $s = \mathrm{fl}(a+b)$ [6, 7]. Os fatos centrais estão na literatura:

- z **é exato.** Se $a \in \mathrm{ulp}(b)\mathbb{Z}$ e o arredondamento é fiel, então $s - a \in \mathbb{F}$, logo $z = s - a$ (Lema 2 de Jeannerod e Zimmermann [12]; Lema 2.5 de Boldo, Graillat e Muller [11], que supõem $e_a \geq e_b$; também com o lema de Sterbenz em [14]).
- **O erro da soma é representável.** Com arredondamento ao mais próximo e sem *overflow*, $(a+b) - \mathrm{fl}(a+b) \in \mathbb{F}$ (Lema 3 de [14]). Isso inclui, como caso particular, o fato de que $c - f \in \mathbb{F}$ (Proposição 10.1).
- **Condições mais fracas.** [12] e [13] enfraquecem as hipóteses de exponente e tratam outros modos de arredondamento; [11] dá o caso de arredondamentos arbitrários.
- **Formalização.** O Fast2Sum e o algoritmo de Veltkamp/Dekker foram formalizados em Coq e Why/Gappa [17]; ver também a tese [18].

Como $f = s - y$ e s é c arredondado para a grade de passo $u = \mathrm{ulp}(y)$, a forma em escada da fórmula (4.1) é uma consequência direta dessas provas. A absorção é descrita, nos textos clássicos, de modo qualitativo, em dígitos: parte dos dígitos do operando menor é descartada, com exemplos numéricos [1].

1.4.2 O que não encontrei

Nas fontes que consegui abrir, nenhuma enuncia $f = \mathrm{fl}(\mathrm{fl}(a+b)-a)$ em função do bit menos significativo e da ordem de grandeza de b, nem a largura $L = \lambda - \nu + 1$ da zona ruidosa, nem limiares em x para x^n. As mesmas fontes não tratam o caso específico de `binary32` nem de x^n.

Limite desta revisão

Isto *não* é uma prova de ineditismo. Não consegui ler o texto completo do *Handbook* [3], do Higham [2], do artigo original de Goldberg [1] nem da tese de Boldo [18]; o Apêndice D registra o que foi de fato lido. Antes de afirmar que a forma explícita é nova, é preciso ler essas obras.

1.5 Escopo e limitações

- Apenas `binary32` e `binary64`, arredondamento ao mais próximo com desempate para par.
- $c > 0$ nos capítulos centrais; $c < 0$ é tratado como variação (Capítulo 9).
- Sem números subnormais, sem *overflow*, sem precisão estendida e sem FMA.
- x^n é tratado como um valor y já calculado e reutilizado; a influência de *como* y é calculado é discutida à parte.

1.6 Organização

A Seção 1.4 acima situa o trabalho na literatura. O Capítulo 2 reúne os fundamentos de ponto flutuante. O Capítulo 3 formula o problema e a metodologia experimental. O Capítulo 4 demonstra os resultados centrais. Os Capítulos 5 a 8 os aplicam, caso a caso. O Capítulo 9 explora variações (outros expoentes, c negativo, forma de calcular x^n), o Capítulo 10 discute remédios, e o Capítulo 11 propõe aplicações didáticas.

1.7 Reprodutibilidade e origem do código

O código parte do notebook `numError01`, de E. R. L. D. Ribas (licença MIT), que define as funções `iSP`/`iDP` e a função `getULP`. Os experimentos deste texto estão no Apêndice A.

A completar: reprodutibilidade

Versões de Python/NumPy usadas, sementes, tempo de execução e um *Makefile* que regenera todas as figuras e tabelas do texto.

Capítulo 2

Fundamentos de ponto flutuante

2.1 Formatos binários

Um número de ponto flutuante binário normalizado com precisão p é

$$y = \pm M \cdot 2^{E-p+1}, \qquad M \in \mathbb{Z}, \quad 2^{p-1} \le M < 2^p,$$

onde E é o expoente e M é a *significando inteiro*. O conjunto desses números, com E no intervalo permitido, será denotado $\mathbb{F}$. A Tabela 2.1 resume os dois formatos deste texto.

Tabela 2.1: Formatos IEEE 754 tratados neste texto.

Formato	p	bits (sinal/expoente/fração)	$E_{\min}$	$E_{\max}$
`binary32`	24	1/8/23	-126	127
`binary64`	53	1/11/52	-1022	1023

2.2 Binades e ULP

Definição 2.1 (binade e ULP)**.** Para $y > 0$ em $\mathbb{F}$, a *binade* de y é o inteiro $E = \lfloor \log_2 y \rfloor$, isto é, $y \in [2^E, 2^{E+1})$. A *ULP* de y é $\operatorname{ulp}(y) = 2^{E-p+1}$, o espaçamento entre y e o próximo número de $\mathbb{F}$ à sua direita.

Em Python, para `binary64`, $\operatorname{ulp}(y)$ = `np.nextafter(y, np.inf) - y`, que é a função `getULP` do notebook de partida. Com $u = \operatorname{ulp}(y)$ vale $y = M \cdot u$, de modo que y *é múltiplo de* u, e todos os números da binade são múltiplos de u.

Observação 2.2. Existem definições diferentes de ULP, que discordam apenas em potências de dois [9]. A usada aqui coincide com `nextafter` acima e com a de Goldberg [1] para y que não é potência de dois.

2.3 Arredondamento ao mais próximo, empate para par

Definição 2.3 (RN)**.** Para $t \in \mathbb{R}$ dentro da faixa de $\mathbb{F}$, $\operatorname{RN}(t)$ é o elemento de $\mathbb{F}$ mais próximo de t. Se t é o ponto médio entre dois elementos vizinhos, $\operatorname{RN}(t)$ é aquele cujo significando inteiro M é par.

Uma operação básica $\circ \in \{+, -, \times, \div\}$ em ponto flutuante devolve $\operatorname{fl}(a \circ b) = \operatorname{RN}(a \circ b)$.

Lema 2.4 (RN em uma binade)**.** *Seja $t \in [2^E, 2^{E+1})$ e $u = 2^{E-p+1}$. Então $\operatorname{RN}(t) = u \cdot N$, onde N é o inteiro mais próximo de t/u, com empate resolvido para N par.*

Proof. Os elementos de $\mathbb{F}$ em $[2^E, 2^{E+1}]$ são exatamente os múltiplos inteiros de u nesse intervalo (o extremo $2^{E+1} = u \cdot 2^p$ também está em $\mathbb{F}$). O mais próximo de t é $u \cdot N$ com N inteiro mais próximo de t/u. Em caso de empate, o significando inteiro de uN é N (ou $N/2$ no extremo, que é par), e a regra do empate escolhe N par. ∎

2.4 Subtração exata: o lema de Sterbenz

Lema 2.5 (Sterbenz)**.** *Se $a, b \in \mathbb{F}$ e $b/2 \le a \le 2b$, então $a - b \in \mathbb{F}$.*

Proof. Suponha $0 < a \le b$ (o outro caso é simétrico, trocando os papéis). Então $\mathrm{ulp}(a) \le \mathrm{ulp}(b)$, e tanto a quanto b são múltiplos de $w = \mathrm{ulp}(a)$. Logo $d = b - a = k\,w$ com k inteiro, e $0 \le d \le b/2 \le a < 2^p w$, portanto $k < 2^p$. Assim d é múltiplo de w com $0 \le k < 2^p$, o que o torna representável (é um número de p bits, possivelmente subnormal ou zero, tratados à parte). ∎

Este lema é clássico [5, 1]; a prova acima é reproduzida para que o texto seja autocontido.

2.5 Exemplos numéricos

Exemplo 2.6 ($2^{53}+1$ e $2^{53}+3$)**.** Em `binary64`, 2^{53} tem $E = 53$ e $u = 2$. O número $2^{53}+1$ cai no ponto médio entre 2^{53} ($M = 2^{52}$, par) e $2^{53}+2$ ($M = 2^{52}+1$, ímpar); logo $\mathrm{fl}(2^{53}+1) = 2^{53}$. Já $2^{53} + 3$ está entre $2^{53} + 2$ (M ímpar) e $2^{53} + 4$ (M par), e $\mathrm{fl}(2^{53} + 3) = 2^{53} + 4$.

Exemplo 2.7 (0,1 em `binary64`)**.** O `double` mais próximo de 0,1 é $3602879701896397/2^{55}$, em hexadecimal `0x1.999999999999ap-4`. O numerador é ímpar, logo $\nu(0{,}1) = -55$ no sentido do Capítulo 4. Em `binary32`, 0,1 vale $13421773/2^{27}$ (`0x1.99999ap-4`) e $\nu = -27$.

A completar: exemplos de caderno

- Conversão decimal → binário passo a passo de 0,1, 1/3 e π nos dois formatos.
- Quadro com a distribuição de ULPs por binade e o ϵ de máquina de cada formato.
- Exemplo de absorção: $10^{16} + 1$ em `binary64`.

Capítulo 3

O problema e a metodologia

3.1 A função

Para $n \in \mathbb{N}$, $x > 0$ e $c > 0$ em $\mathbb{F}$, seja $y = \mathrm{fl}(x^n)$ e

$$f(x) = \mathrm{fl}\big(\mathrm{fl}(y + c) - y\big). \tag{3.1}$$

Matematicamente $f \equiv c$. O texto trabalha principalmente com $n = 10$.

3.2 A observação inicial

No notebook de partida, a função aparece com $c = 1$ em torno de $x \in [39{,}2;\ 42{,}4]$, onde o gráfico mostra duas faixas de valores (0 e 2) no lugar da reta $f = 1$. A primeira pergunta é por que esse intervalo, e a segunda é o que acontece com outros c.

A completar: reproduzir o gráfico inicial

Incluir o gráfico de $c = 1$ em $[39{,}2;\ 42{,}4]$ e a sequência dos exemplos 01–06 do notebook, com legenda explicando cada um à luz do Capítulo 4.

3.3 Hipóteses

As hipóteses abaixo valem em todo o texto, salvo quando dito o contrário.
H1 Arredondamento ao mais próximo, empate para par, em todas as operações.
H2 As duas ocorrências de x^n produzem o mesmo valor y.
H3 Nenhuma operação é fundida (sem FMA) e nenhuma usa precisão estendida.
H4 $c \in \mathbb{F}$ é positivo e não há *overflow* nem *underflow*.

A completar: quando H2 e H3 falham

Exemplos concretos: compiladores que reordenam a expressão, unidades x87 com 80 bits, vetorização, `pow` de bibliotecas diferentes.

3.4 Como $y = x^n$ é calculado

A completar: `pow` contra produtos

Comparar `x**10`, `x*x*...*x` e quadrados sucessivos: erro de y em ULPs para $x \in [30, 80]$ e efeito sobre x_1, x_2 e L. Resultado esperado: a teoria vale para qualquer y efetivamente

calculado, e só muda o conjunto de x associado a cada binade.

3.5 Metodologia experimental

- **Software.** NumPy; em `binary32`, x e c são `np.float32` para evitar promoção a `float64`.
- **Amostragem por binade.** Para cada c e cada binade E perto da zona prevista, sorteiam-se 1 500 a 4 000 valores de x uniformes em $[2^{E/n}, 2^{(E+1)/n})$, agrupados pelo expoente real de y.
- **Valores de c.** 1 001 inteiros ímpares (1 a 2 001), 300 frações diádicas e 310 não diádicas (0,1; 0,2; 0,3; 1/3; π; e; 300 reais em [0,01; 50]), em cada precisão.
- **Validação.** O conjunto de valores observados de f em cada binade é comparado com o conjunto previsto pela Proposição 4.1.
- **Erro em ULPs.** $\varepsilon = (f - c)/\mathrm{ulp}(y)$.

A completar: protocolo estatístico

Justificar o tamanho das amostras, tratar a densidade de y dentro da binade (que não é uniforme quando x é uniforme) e relatar intervalos de confiança para as frequências do efeito de borda.

Capítulo 4

Teoria: o modelo da ULP

Este capítulo contém o núcleo do texto. Ele estabelece, sob as hipóteses H1–H4, uma fórmula para $f(x)$ em cada binade de y.

4.1 Notação

Seja $c \in \mathbb{F}$, $c > 0$. Escreva $c = m \cdot 2^{\nu}$ com m ímpar; então $1 \le m < 2^{p}$. Chamamos $\nu = \nu(c)$ de *expoente do menor bit* de c: é 0 para inteiro ímpar, $-j$ para $m/2^{j}$, e -55 para o `double` 0,1. Escreva também $\lambda = \lfloor \log_2 c \rfloor$.

Para $y \in [2^{E}, 2^{E+1})$ e $u = \operatorname{ulp}(y) = 2^{E-p+1}$, seja $q = c/u$ e $M = y/u$ (inteiro). Definimos

$$E_1 = p + \nu, \qquad E_2 = p + \lambda, \qquad L = E_2 - E_1 + 1 = \lambda - \nu + 1.$$

4.2 A proposição principal

Proposição 4.1. *Sejam $y \ge c > 0$ e $u = 2^{E-p+1}$, e suponha $y + c < 2^{E+1}$ (a soma permanece na binade de y). Então*

$$f = \begin{cases} c, & u \le 2^{\nu}, \\ u\,\lfloor q + \tfrac{1}{2} \rfloor, & u > 2^{\nu} \ \text{e}\ q \notin \mathbb{Z} + \tfrac{1}{2}, \\ u\,k, \quad k \in \{\lfloor q \rfloor, \lceil q \rceil\},\ k \equiv M \pmod 2, & q \in \mathbb{Z} + \tfrac{1}{2}. \end{cases} \tag{4.1}$$

Além disso, o terceiro caso ocorre se e somente se $u = 2^{\nu+1}$.

Proof. Passo 1: a subtração é exata. Como $y \ge c$, temos $y \le y + c \le 2y$. O número $s = \mathrm{fl}(y + c)$ satisfaz $y \le s \le 2y$, pois $y \in \mathbb{F}$ e RN é monótona. Pelo Lema 2.5, $s - y \in \mathbb{F}$, logo $f = \mathrm{fl}(s - y) = s - y$.

Passo 2: arredondamento da soma. Como $y + c \in [2^{E}, 2^{E+1})$, o Lema 2.4 dá $s = u\,N$, com N o inteiro mais próximo de $(y + c)/u = M + q$, e empate resolvido para N par. Como M é inteiro, $N = M + k$ com k o inteiro mais próximo de q, e o empate (se $q \in \mathbb{Z} + \tfrac{1}{2}$) é resolvido por $M + k$ par, isto é, $k \equiv M \pmod 2$. Portanto $f = s - y = u\,k$.

Passo 3: os casos. Se $q \notin \mathbb{Z} + \tfrac{1}{2}$, o inteiro mais próximo de q é $\lfloor q + \tfrac{1}{2} \rfloor$, o que dá o segundo caso quando $q \notin \mathbb{Z}$. Se $u \le 2^{\nu}$, então u divide 2^{ν} e portanto divide $c = m2^{\nu}$; logo $q \in \mathbb{Z}$, $k = q$ e $f = c$. Se $u = 2^{e} > 2^{\nu}$, então $q = m\,2^{\nu-e}$ com m ímpar e $\nu - e \le -1$. Esse número é semi-inteiro se $\nu - e = -1$ (pois $m/2 \in \mathbb{Z} + \tfrac{1}{2}$) e não é inteiro nem semi-inteiro se $\nu - e \le -2$ (pois a parte fracionária de $m/2^{j}$, $j \ge 2$, tem denominador $2^{j} \ge 4$). Isso mostra que o terceiro caso ocorre se e somente se $u = 2^{\nu+1}$, e que $f \ne c$ sempre que $u > 2^{\nu}$. ∎

Proposição : demonstrada e verificada

Demonstrada acima sob H1–H4 e $y + c < 2^{E+1}$. Verificada experimentalmente em 1 611 valores de c por precisão (Capítulo 3), com a regra de paridade confirmada em 4×10^6 amostras sem exceção.

Observação 4.2 (Relação com o Fast2Sum). O Passo 1 acima é o resultado clássico de que $z = \mathrm{fl}(s - a)$ é exato no Fast2Sum, aqui obtido diretamente do lema de Sterbenz, porque $y \le s \le 2y$. Em termos de [12, 11], a condição é $y \in \mathrm{ulp}(c)\mathbb{Z}$ (isto é, y é múltiplo da ULP de c), que vale porque $y \ge c$. O Passo 2 é a descrição de s como c arredondado à grade de passo u. O que a Proposição 4.1 acrescenta é a leitura em função de $\nu(c)$ e $\lfloor \log_2 c \rfloor$ (Teorema 4.4); ver a Seção 1.4.

Exemplo 4.3 ($x = 40$, $c = 1, 3, 5$ em `binary64`)**.** Aqui $y = 40^{10} = 2^{30} \cdot 5^{10} = 10\,485\,760\,000\,000\,000$, exatamente representável. Como $2^{53} \approx 9{,}007 \times 10^{15} \le y < 2^{54}$, temos $E = 53$ e $u = 2$, com $M = y/2 = 5\,242\,880\,000\,000\,000$, par. Para $c = 1$, $q = \frac{1}{2}$ e $k \in \{0, 1\}$ com $k \equiv M$, logo $k = 0$ e $f = 0$. Para $c = 3$, $q = \frac{3}{2}$ e $k \in \{1, 2\}$, logo $k = 2$ e $f = 4$. Para $c = 5$, $q = \frac{5}{2}$ e $k \in \{2, 3\}$, logo $k = 2$ e $f = 4$. Os três valores coincidem com os calculados em Python. Em um x vizinho com M ímpar a regra daria $f = 2$, 2, 6.

4.3 Limiares e largura da zona ruidosa

Teorema 4.4. *Sob as hipóteses da Proposição 4.1 e com $\nu \le \lambda$:*

(i) Se $y < 2^{p+\nu}$ (isto é, $E < E_1$), então $f = c$.

(ii) Se $E_1 \le E \le E_2$, então $f \ne c$. Em $E = E_1$, $f \in \{c - u/2,\, c + u/2\}$; em $E_1 < E \le E_2$, $f = u\lfloor q + \frac{1}{2} \rfloor$ é constante na binade.

(iii) Se $E > E_2$, então $f = 0$.

Em termos de x, com $y \approx x^n$, a zona ruidosa é $x_1 \le x < x_2$, onde

$$x_1 = 2^{(p+\nu)/n}, \qquad x_2 = 2^{(p+1+\lambda)/n}, \tag{4.2}$$

e seu número de binades é $L = \lambda - \nu + 1$, o comprimento em bits de m (o número de bits significativos *de c).*

Proof. (i) $E < E_1 = p + \nu$ equivale a $E - p + 1 \le \nu$, isto é, $u \le 2^\nu$. O primeiro caso de (4.1) dá $f = c$. A hipótese $y \ge c$ é parte da Proposição.

(ii) Para $E \ge E_1$ temos $u > 2^\nu$, logo $f \ne c$ pela prova da Proposição. Também $y \ge 2^{p+\nu} > m2^\nu = c$, porque $m < 2^p$; portanto $y \ge c$ vale automaticamente nessa faixa. Em $E = E_1$, $u = 2^{\nu+1}$ e $q = m/2 \in \mathbb{Z} + \frac{1}{2}$, o terceiro caso; os dois valores possíveis de k são $\lfloor q \rfloor$ e $\lceil q \rceil$, e $uk = c \mp u/2$. Fora de E_1 não há empate e $f = u\lfloor q + \frac{1}{2} \rfloor$ depende só de E. Falta ver que $f \ne 0$ se e somente se $E \le E_2$. Temos $f = 0$ se e somente se $\lfloor q + \frac{1}{2} \rfloor = 0$, isto é, $q < \frac{1}{2}$, isto é, $u > 2c$. Se c não é potência de dois, $u = 2^{E-p+1}$ nunca é igual a $2c$, e $u > 2c \iff E - p + 1 \ge \lambda + 2 \iff E > E_2$. Se $c = 2^\lambda$ ($m = 1$), em $E = E_2$ temos $u = 2c$, $q = \frac{1}{2}$ e $E_2 = E_1$: é a binade de empate, e $f \in \{0, 2c\}$ não é identicamente nulo. Em ambos os casos, $f = 0$ identicamente apenas para $E > E_2$.

(iii) Segue do argumento anterior.

Os limites em x vêm de $y \approx x^n$: $E = E_1$ corresponde a $x \approx 2^{E_1/n}$ e $E = E_2 + 1$ a $x \approx 2^{(E_2+1)/n}$. Por fim, $\lfloor \log_2 m \rfloor = \lambda - \nu$, e o comprimento em bits de m é $\lfloor \log_2 m \rfloor + 1 = L$. ∎

Corolário 4.5 (largura em x)**.** $x_2/x_1 = 2^{L/n}$. *Para $c = 2^j$ ($L = 1$) isso dá $2^{1/n}$; para $n = 10$,* $\approx 1{,}07$.

Corolário 4.6 (bits de c e `tz`)**.** *Se $c \in \mathbb{F}$ tem significando inteiro M_c ($2^{p-1} \le M_c < 2^p$) com τ zeros à direita, então $L = p - \tau$. Em particular, c não diádico típico tem τ pequeno e L próximo de p.*

Proof. $m = M_c/2^\tau$ tem $p - \tau$ bits. ∎

4.4 Erro em ULPs

Corolário 4.7. *Sob as hipóteses da Proposição 4.1, em toda binade com $u > 2^\nu$,*

$$\varepsilon = \frac{f - c}{u} = k - q \in \left[-\tfrac{1}{2}, \tfrac{1}{2}\right],$$

com $\varepsilon = \pm\frac{1}{2}$ exatamente na binade de empate E_1. Em particular $|f - c| \le u/2$ e o erro relativo é $|f - c|/c \le u/(2c)$, que atinge 100% quando $f = 0$.

Proof. k é um inteiro mais próximo de q; logo $|k - q| \le \frac{1}{2}$, com igualdade só quando q é semi-inteiro, isto é, em E_1. ∎

4.5 Quantos valores f assume em uma binade

Corolário 4.8. *Em cada binade $E \neq E_1$ (sem cruzamento de binade), f assume um único valor. Em E_1, assume exatamente dois, $c \pm u/2$, ambos atingidos em qualquer sequência de dois números de $\mathbb{F}$ consecutivos na binade.*

Proof. A primeira afirmação é a Proposição 4.1. Na segunda, k depende apenas da paridade de M, e M alterna de paridade ao longo dos números consecutivos da binade. ∎

4.6 Efeito de borda de binade

A Proposição 4.1 supõe $y + c < 2^{E+1}$.

Lema 4.9 (borda)**.** *Se $y \ge c$ e $y + c \ge 2^{E+1}$, então $|f - c| \le \mathrm{ulp}(y)$.*

Proof. Pelo Lema 2.5, $f = s - y$ com $s = \mathrm{fl}(y + c)$, de modo que $f - c = s - (y + c)$. Como $y + c \ge 2^{E+1}$, os números de $\mathbb{F}$ próximos de $y + c$ têm espaçamento $2u$ (ou $y + c$ está em $[2^{E+1}, 2^{E+1} + 2u)$, onde o vizinho à esquerda $2^{E+1} - u$ está mais longe do que 2^{E+1}). Logo $|s - (y + c)| \le u$. ∎

Observação 4.10 (frequência). Se o significando M de y fosse uniforme na binade, a fração de y com $y + c \ge 2^{E+1}$ seria $c/2^E$, e metade delas produziria desvio não nulo; a frequência de desvios seria, portanto, da ordem de $c/2^{E+1} \approx c/(2y)$. É isso que se observa.

Efeito de borda

O Lema 4.9 está demonstrado. A *frequência* é heurística: mediu-se, em `binary64`, 0 desvios em 4×10^7 amostras da zona ruidosa para c ímpar ou diádico; 0,3% para c não diádico; em `binary32`, 82 em 4×10^7 para c ímpar e 1% para c não diádico.

4.7 A região $y < c$

Lema 4.11. *Se $0 < y < c$ e $c \in [2^a, 2^{a+1})$, então $|f - c| \le 2\,\mathrm{ulp}(c)$.*

Esboço. Seja $w = \mathrm{ulp}(c)$ e $t = y + c \in (c, 2c) \subset [2^a, 2^{a+2})$. Os números de $\mathbb{F}$ nessa faixa têm espaçamento w ou $2w$, logo $s = \mathrm{fl}(t)$ satisfaz $|s - t| \le w$. O valor exato $s - y = c + (s - t)$ está em $[c - w, c + w]$, onde o espaçamento é no máximo $2w$, e o arredondamento de $s - y$ introduz outro erro de no máximo w. A soma dá $|f - c| \le 2w$. ■

Região $y < c$

A cota $2\,\mathrm{ulp}(c)$ está demonstrada (esboço). Mediu-se $|f-c| \le \mathrm{ulp}(c)$ em todas as amostras (2% a 23% das amostras com $f \neq c$). A cota $1\,\mathrm{ulp}(c)$ *não* está demonstrada.

4.8 Generalização a n e p

Os resultados valem para qualquer p e qualquer y calculado. O expoente n só entra na conversão de binades em x, por (4.2). Para $n = 23$ e $p = 53$, $c = 1$, $x_1 = 2^{53/23} \approx 4{,}94$, o que explica as janelas $x \in [4, 6]$ no notebook de partida.

4.9 O que está e o que não está demonstrado

Demonstrado (H1–H4)	Proposição 4.1, Teorema 4.4, Corolários 4.6 e 4.7, Lema 4.9, Lema 4.11 (cota 2 ulp)
Só verificado	Frequência do efeito de borda; cota $1\,\mathrm{ulp}(c)$ para $y < c$
Não coberto	$c < 0$; c da ordem de y em que $y < c$ domina; FMA; precisão estendida; qualidade de `pow`

A completar: demonstrações adicionais

- Prova da cota $1\,\mathrm{ulp}(c)$ na região $y < c$ (ou contraexemplo).
- Fórmula fechada da frequência do efeito de borda, com a densidade de y induzida por x uniforme.
- Extensão a $c < 0$ com $|c| \le y/2$.

Capítulo 5

Caso base: c potência de dois

Para $c = 2^j$, tem-se $m = 1$, $\nu = \lambda = j$ e $L = 1$. O Teorema 4.4 diz que existe uma única binade ruidosa, $E_1 = E_2 = p + j$, onde $f \in \{0,\, 2^{j+1}\}$, e depois $f = 0$.

$$x_1 = 2^{(p+j)/n}, \qquad x_2 = x_1 \cdot 2^{1/n}.$$

Para $c = 1$ e $n = 10$, $p = 53$: $x_1 \approx 39{,}40$ e $x_2 \approx 42{,}22$, o que coincide com o intervalo do notebook de partida, [39,2; 42,4].

Exemplo 5.1 ($c = 1$ em $x = 40$)**.** Pelo Exemplo 4.3, $f(40) = 0$; em um x vizinho com M ímpar, $f = 2$.

A completar

- Resumo dos resultados anteriores sobre potências de dois (referenciar o trabalho já existente) e verificação de que são casos particulares do Teorema 4.4.
- Tabela de x_1, x_2 para $c = 2^j$, $j = -10, \ldots, 20$, nos dois formatos.
- Figura: $f(x)$ para $c = 1$, 2, 4, 0,5, mostrando que a faixa dupla desloca-se com j.

Capítulo 6

Inteiros ímpares

6.1 O limiar universal

Para c ímpar, $\nu = 0$ e $E_1 = p$. Em `binary64`, o erro começa sempre em $x_1 = 2^{53/10} \approx 39{,}3966$, onde $y = 2^{53}$ e a ULP passa de 1 para 2. O limiar não depende de c. A zona termina em $x_2 = 2^{(54+\lfloor \log_2 c \rfloor)/10}$, e tem $L = \text{bitlen}(c)$ binades.

6.2 A binade de empate

Em $E = 53$ ($u = 2$), c ímpar fica exatamente entre dois múltiplos de 2: $f \in \{c-1, c+1\}$, conforme a paridade de $M = y/2$. Para $c = 1$ o resultado é $\{0, 2\}$; o Exemplo 4.3 mostra $f(40) = 0$, 4 e 4 para $c = 1, 3, 5$.

6.3 Os degraus

Nas binades seguintes $f = u\lfloor c/u + \frac{1}{2} \rfloor$ é constante, e o erro é

$$f - c = \begin{cases} -(c \bmod u), & (c \bmod u) < u/2, \\ u - (c \bmod u), & (c \bmod u) > u/2. \end{cases}$$

Exemplo 6.1 ($c = 7$)**.** $E = 53$: $u = 2$, $f \in \{6, 8\}$. $E = 54$: $u = 4$, $q = 1{,}75$, $f = 8$. $E = 55$: $u = 8$, $q = 0{,}875$, $f = 8$. $E = 56$: $u = 16$, $q = 0{,}4375 < \frac{1}{2}$, $f = 0$. A zona é $E = 53, 54, 55$ ($L = 3 = \text{bitlen}(7)$), de $x_1 = 39{,}40$ a $x_2 = 48{,}50$. Por exemplo $f(45) = 8$ (Python confirma).

Exemplo 6.2 ($c = 101$)**.** A Tabela 6.1 detalha o caso, e a Figura 6.1 o mostra junto de $c = 3, 5, 173$.

Duas observações. O erro em ULPs é pequeno, mas o relativo não: em $c = 101$ o resultado chega a 128 (27%) e depois a 0 (100%). E f não é monótona em x: o sinal do erro em cada degrau depende dos bits de c.

6.4 O mapa (x, c)

A Figura 6.2 mostra, para os ímpares de 1 a 255, o erro relativo em função de x. A fronteira do preto ($f = 0$) coincide com $x_2(c)$ do Teorema 4.4, e os saltos de x_2 ocorrem em $c = 2^k$.

Tabela 6.1: Valores de f por binade para $c = 101$ (`binary64`).

E	x	u	c/u	$f(x)$	$\varepsilon = (f - c)/u$
< 53	$< 39{,}40$	≤ 1	≥ 101	101	0
53	39,40 a 42,22	2	50,5	100 ou 102	−0,5 ou +0,5
54	42,22 a 45,25	4	25,25	100	−0,25
55	45,25 a 48,50	8	12,63	104	+0,375
56	48,50 a 51,98	16	6,31	96	−0,31
57	51,98 a 55,72	32	3,16	96	−0,16
58	55,72 a 59,71	64	1,58	128	+0,42
59	59,71 a 64,00	128	0,79	128	+0,21
≥ 60	$\geq 64{,}00$	≥ 256	$\leq 0{,}39$	0	−0,39 a 0

Tabela 6.2: Limites da zona ruidosa para c ímpar (`binary64`).

c	L	x_1	x_2
1	1	39,40	42,22
3	2	39,40	45,25
5 e 7	3	39,40	48,50
101	7	39,40	64,00
1 001	10	39,40	78,79
65 537	17	39,40	128,00

A completar: mais exemplos e demonstrações

- $c = 2^k \pm 1$ e $c = 2^k - 1$ (por exemplo 255 e 1023): a sequência de degraus é quase constante?
- c com muitos zeros à direita (por exemplo 96 e 1000): $\nu > 0$ e x_1 deslocado; contraste com o caso ímpar.
- Contagem: quantos dos degraus de f ficam acima de c e quantos abaixo? Distribuição do sinal do erro em função dos bits de c.
- Demonstração de que a sequência $\varepsilon_E = \lfloor q_E + \frac{1}{2} \rfloor - q_E$, com $q_E = c/2^{E-p+1}$, está ligada à expansão binária de c (cada degrau examina um bit a mais).
- Teorema sobre o *valor* de f no último degrau não nulo: $f = 2^{\lambda+1}$ para c não potência de dois.

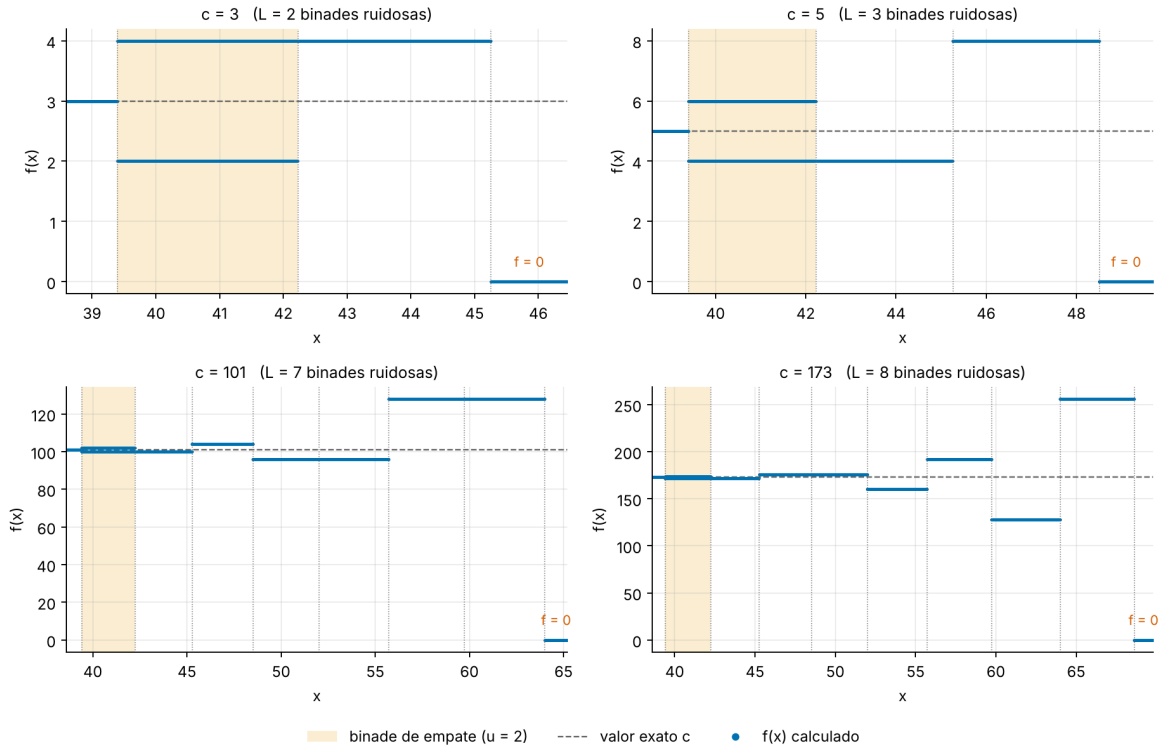


Figura 6.1: $f(x)$ em `binary64` para $c = 3, 5, 101$ e 173. Faixa laranja: binade de empate ($u = 2$). Linhas pontilhadas: fronteiras de binade $x = 2^{E/10}$. O traço tracejado é o valor exato c.

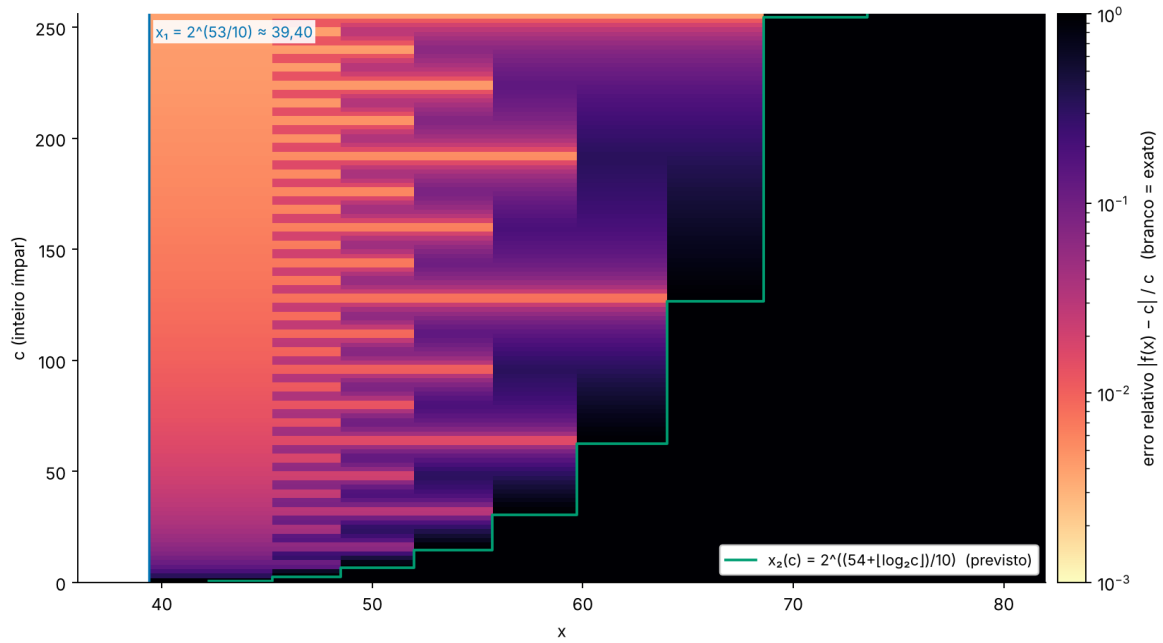


Figura 6.2: Erro relativo $|f - c|/c$ para os inteiros ímpares de 1 a 255 (linhas) e x de 36 a 82 (colunas). Branco: exato. Preto: $f = 0$. Linha verde: $x_2(c)$ previsto.

Capítulo 7

Frações

7.1 Frações diádicas

Para $c = m/2^j$ (m ímpar, $j > 0$), $\nu = -j$. O Teorema 4.4 dá $x_1 = 2^{(53-j)/10}$ e $L = \text{bitlen}(m)$.

Exemplo 7.1. $c = 3/4$ ($j = 2$): $x_1 = 34{,}30$, $x_2 = 39{,}40$, $L = 2$; o erro relativo é $\pm 1/3$ na binade de empate e depois -1. $c = 5/16$ ($j = 4$): $x_1 = 29{,}86$, $x_2 = 36{,}76$, $L = 3$.

A zona de $c = m/2^j$ é a zona de $c = m$ deslocada em j/n no expoente de x: ela *começa antes* porque a ULP de y precisa ser menor para que c ainda esteja na grade.

7.2 Frações não diádicas

7.2.1 O que significa $c = 0{,}1$

O programa não sabe de 0,1: ele usa $c \in \mathbb{F}$ mais próximo, que para `binary64` é $3602879701896397/2^{55}$ (Exemplo no Capítulo 2). Todo o texto trata de $c \in \mathbb{F}$; a diferença para o decimal original é uma segunda fonte de erro, de no máximo $\frac{1}{2}\text{ulp}(c)$, discutida adiante.

7.2.2 Zona larga

Pelo Corolário 4.6, $L = p - \tau$, onde τ é o número de zeros à direita do significando inteiro de c. Para 0,1, o significando é terminado em `...a` (hexadecimal), $\tau = 1$ e $L = 52$; para 1/3, $\tau = 0$ e $L = 53$; para π (`0x1.921fb54442d18p+1`), $\tau = 3$ e $L = 50$. Um c aleatório tem τ geométrico, com média próxima de 1, e portanto $L \approx p - 1$. A zona ocupa quase todo o intervalo $x \in \left[2^{(1+\lambda)/10}, 2^{(54+\lambda)/10}\right]$ em `binary64`. Para $c = 0{,}1$ ($\lambda = -4$) isso dá $x \in [0{,}87; 32]$.

7.2.3 Crescimento do erro

A ULP de x^n é proporcional a x^n; logo o erro relativo $\lesssim u/(2c)$ é multiplicado por 2^n a cada duplicação de x (por 1 024 para $n = 10$).

A completar

- Erro de representação de c (decimal → binário) contra erro aritmético: em que x um domina o outro?
- Demonstrar $\mathbb{E}[L] = p - 1 + 2^{1-p} \approx p - 1$ para c com significando uniforme.
- Tabela de L para constantes comuns: 0,1, 0,2, 0,3, 0,5, π, e, $\sqrt{2}$, $\ln 2$.
- Efeito de somar uma fração em loops: N somas de 0,1 contra a fórmula fechada.

Tabela 7.1: Erro relativo $(f - c)/c$ medido em `binary64`.

x	$c = 0{,}1$	$c = 1/3$
1	$+8{,}3 \times 10^{-16}$	$-1{,}7 \times 10^{-16}$
2	$-9{,}1 \times 10^{-13}$	$-2{,}3 \times 10^{-13}$
5	$-3{,}7 \times 10^{-9}$	$+1{,}9 \times 10^{-9}$
10	$+3{,}8 \times 10^{-6}$	$+1{,}9 \times 10^{-6}$
20	$-3{,}9 \times 10^{-3}$	$+2{,}0 \times 10^{-3}$
30	$+0{,}25$	$+0{,}125$
32,5	-1	$-0{,}25$

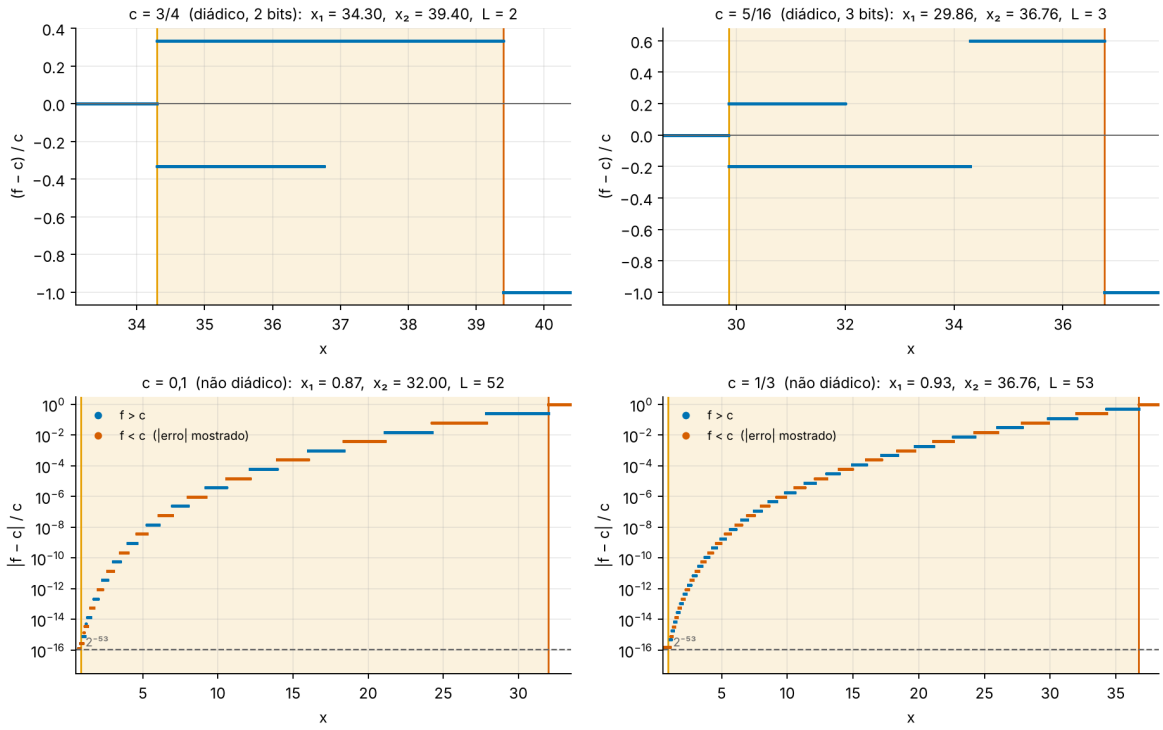


Figura 7.1: Erro relativo em frações diádicas (em cima, escala linear) e não diádicas (embaixo, $|f - c|/c$ em escala logarítmica; vermelho: $f < c$). Faixa laranja: zona ruidosa $[x_1, x_2]$ prevista.

- Comparação com o caso decimal (`decimal.Decimal`).

Capítulo 8

`binary32` contra `binary64`

A teoria depende da precisão p apenas por (4.2). Para c ímpar, $x_1 = 2^{p/10}$: 39,40 em `binary64`, 5,28 em `binary32`. A largura L é a mesma nos dois formatos para c inteiro ou diádico representável nos dois, e passa a ser ≤ 24 para c não diádico em `binary32`.

Tabela 8.1: Zona ruidosa prevista $[x_1, x_2)$ e L para $n = 10$.

	`binary64`			`binary32`		
c	x_1	x_2	L	x_1	x_2	L
1	39,40	42,22	1	5,28	5,66	1
3	39,40	45,25	2	5,28	6,06	2
5 e 7	39,40	48,50	3	5,28	6,50	3
101	39,40	64,00	7	5,28	8,57	7
1 001	39,40	78,79	10	5,28	10,56	10
1/2	36,76	39,40	1	4,92	5,28	1
3/4	34,30	39,40	2	4,59	5,28	2
0,1	0,87	32,00	52	0,81	4,29	24
1/3	0,93	36,76	53	0,93	4,92	24

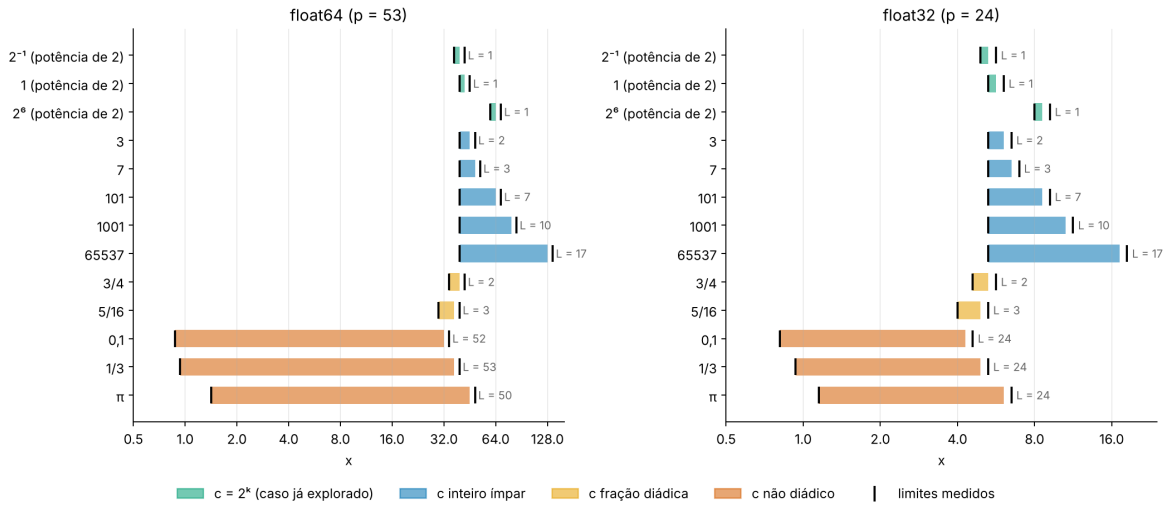


Figura 8.1: Zona ruidosa prevista $[x_1, x_2]$ (barras) e limites medidos (traços pretos) para valores de c de cada classe, em `binary64` e `binary32`. L = número de binades da zona.

8.1 Validação: o dente de serra

A Figura 8.2 reúne 2 913 pares (q, ε) medidos em `binary64`. Todos caem sobre $\varepsilon = \mathrm{arred}(q) - q$, com $|\varepsilon| \leq \frac{1}{2}$, como prevê o Corolário 4.7.

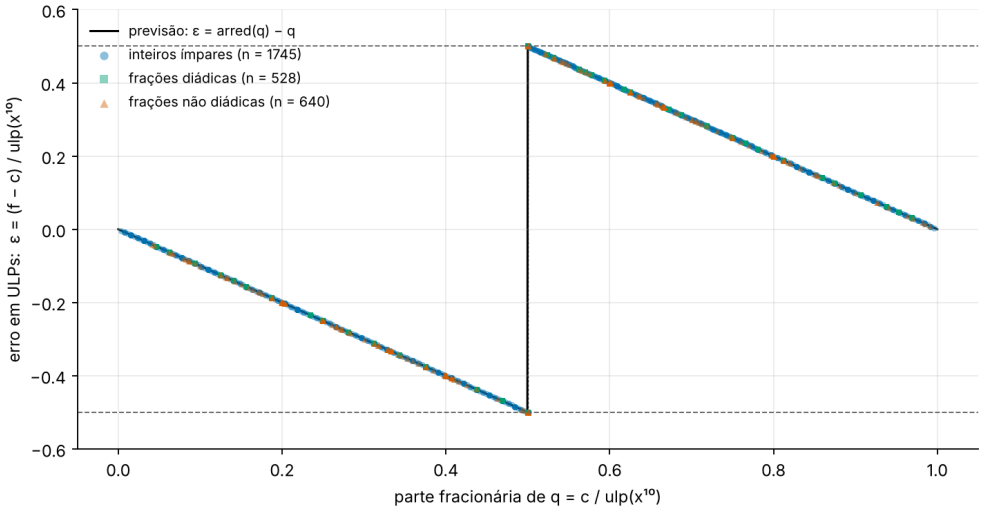


Figura 8.2: Erro em ULPs, $\varepsilon = (f - c)/\mathrm{ulp}(x^{10})$, contra a parte fracionária de $q = c/\mathrm{ulp}(x^{10})$, em `binary64`. A linha preta é a previsão $\varepsilon = \mathrm{arred}(q) - q$.

A completar

- O mesmo dente de serra para `binary32`.
- Quando c é representável em `binary64` mas não em `binary32` (por exemplo $c = 2^{24} + 1$): a comparação deixa de ser justa; discutir a conversão de c.
- Tabela de erros de f *medidos* em x fixo ($x = 5, 6, 8$ em `binary32`; $x = 40, 50, 64$ em `binary64`) para uma lista de c.
- Mistura de precisões: x em `binary32` e c em `binary64` (promoção implícita) e o efeito sobre a zona.

Capítulo 9

Variações

9.1 Outros expoentes n

O expoente só entra na conversão de binade em x: $x_1 = 2^{(p+\nu)/n}$ e $x_2 = 2^{(p+1+\lambda)/n}$. A Tabela 9.1 dá x_1 para c ímpar ($\nu = 0$). O caso $n = 23$ explica o intervalo $x \in [4, 6]$ dos exemplos do notebook de partida.

Tabela 9.1: Limiar $x_1 = 2^{p/n}$ para c ímpar.

n	`binary64`	`binary32`
2	$9{,}49 \times 10^7$	4 096
3	$2{,}08 \times 10^5$	256
5	1 552	27,9
10	39,40	5,28
23	4,94	2,06

A completar

Verificar numericamente $n = 2, 3, 5, 23$ (validação análoga à do Capítulo 3) e discutir n não inteiro (`x**1.5`).

9.2 c negativo

A completar

Com $c < 0$ e $|c| \leq y/2$, o lema de Sterbenz ainda dá subtração exata ($s = \mathrm{fl}(y + c) \in [y/2, y]$), mas a soma pode cruzar a fronteira inferior da binade, onde o espaçamento *cai* para a metade. Estudar se a frequência do efeito de borda aumenta e se a fórmula fechada ganha uma segunda forma. Hipótese de trabalho: $f(x; -c) = -f(x; c)$ em quase toda a zona, exceto por uma classe de empates diferente.

9.3 c da mesma ordem de y

A completar

Região $y < c$ e $y \approx c$: provar a cota $\mathrm{ulp}(c)$ (Lema 4.11) e descrever o que ocorre quando a subtração não é exata ($y < c/2$).

9.4 Como y é calculado

A completar

Medir, para $x \in [30, 80]$, a diferença em ULPs entre `x**10`, produtos sucessivos e quadrados sucessivos, e mostrar que a Proposição 4.1 continua valendo para o y efetivo, com x_1 e x_2 praticamente inalterados.

9.5 FMA e precisão estendida

A completar

Em hardware com FMA, a expressão $y + c - y$ pode ser compilada como $\mathrm{fma}(\cdot)$ ou reordenada; em x87, os 80 bits intermediários fazem $f \equiv c$ em quase toda a zona. Reproduzir com um pequeno programa em C compilado de duas formas.

Capítulo 10

Remédios

10.1 Reescrever a expressão

A forma $(x^n - x^n) + c$ ou, simplesmente, c, elimina o problema. Para expressões em que o cancelamento não é algébrico, isso não é possível; os métodos abaixo atuam sobre o valor.

10.2 O resultado é o z do Fast2Sum

O algoritmo *Fast2Sum* de Dekker [6], para $a, b \in \mathbb{F}$ com exponente de a maior ou igual ao de b, calcula

$$s = \mathrm{fl}(a + b), \qquad z = \mathrm{fl}(s - a), \qquad t = \mathrm{fl}(b - z),$$

e garante $s + t = a + b$ exatamente, sob arredondamento ao mais próximo. Com $a = y$ e $b = c$ (válido, pois $y \geq c$), o valor z é exatamente $f(x)$ da equação (3.1). Logo $t = \mathrm{fl}(c - f)$, e o *erro de f é representável e computável.*

Proposição 10.1 (conhecida)**.** *Sob as hipóteses da Proposição 4.1, $c - f \in \mathbb{F}$.*

Este é um caso particular do fato geral de que o erro de uma adição com arredondamento ao mais próximo é representável (Lema 3 de [14]; ver também [11, 12]). A prova abaixo é direta e específica, e mostra *quantos* bits o erro usa.

Proof. Se $E > E_2$, $f = 0$ e $c - f = c \in \mathbb{F}$. Se $E \leq E_2$, pelo Corolário 4.7, $|c - f| \leq u/2 = 2^{E-p}$. Além disso, $c - f$ é múltiplo de 2^ν (diferença de dois múltiplos de 2^ν, já que $u > 2^\nu$ na zona ruidosa e $f = uk$). Assim $|c - f| = j\,2^\nu$ com $j \leq 2^{E-p-\nu} \leq 2^{E_2-p-\nu} = 2^{\lambda-\nu} = 2^{L-1} < 2^p$. Todo número da forma $j\,2^\nu$ com $|j| < 2^p$ pertence a $\mathbb{F}$. ∎

O fato geral é parte da teoria das transformações livres de erro [6, 7]; a presença dele no *Handbook* [3] não foi conferida diretamente. O ganho aqui é que o erro é conhecido explicitamente, pela fórmula (4.1).

10.3 Soma compensada

A completar

- Algoritmo de Kahan [8] aplicado a $y + c - y$ e a séries de termos.
- TwoSum de Knuth [7] (sem restrição de ordem).
- `math.fsum` e `numpy.sum` (soma em pares): onde ajudam e onde não.
- Experimento: fração de x na zona ruidosa em que cada método recupera c exatamente.

Capítulo 11

Aplicações didáticas

O resultado dá um roteiro para uma oficina: *escreva c em binário, ache ν e $\lfloor \log_2 c \rfloor$, e preveja o gráfico de f.*

11.1 Roteiro de oficina

A completar

1. Aquecimento: $2^{53}+1$ no Python (`9007199254740993.0 == 9007199254740992.0`).
2. A grade de ULPs: `np.nextafter` em $y = 1$, 2^{20}, 2^{53}.
3. Previsão em grupos: dado c, prever x_1, x_2, L; comparar com o gráfico.
4. Fecho: por que o erro é sempre $\leq \frac{1}{2}$ ULP, e onde o erro relativo explode.

Duração sugerida e adaptação para estudantes do ensino médio (potências de 2 e divisão com resto) e para engenharia (cancelamento e condicionamento).

11.2 Exercícios com gabarito

Os valores abaixo foram conferidos em Python (`binary64`).

1. Para $c = 7$ e $x = 45$, determine a binade de y, a ULP e $f(45)$.
   Resposta: $E = 54$, $u = 4$, $q = 1{,}75$, $f = 8$.
2. Para $c = 101$ e $x = 50$, determine $f(50)$.
   Resposta: $x \in [48{,}50;\ 51{,}98)$, $E = 56$, $u = 16$, $q \approx 6{,}31$, $f = 96$.
3. Para $c = 0{,}1$ e $x = 20$, determine $f(20)$ e o erro relativo.
   Resposta: $f = 0{,}099609375$; erro relativo $-3{,}9 \times 10^{-3}$.
4. Qual é o menor c ímpar para o qual $f(x) \neq 0$ ainda em $x = 100$?
   Resposta: precisa de $2^{(54+\lambda)/10} > 100$, isto é, $\lambda \geq 13$, $c \geq 8\,193$ (o primeiro ímpar com $\lambda = 13$). Conferência: para $c = 8\,193$, $f(103) = 16\,384$ e $f(104) = 0$ (pois $x_2 = 2^{6{,}7} \approx 103{,}97$).

A completar

Mais exercícios, em três níveis (aplicação direta, demonstração, programação), com gerador automático de enunciados e gabaritos.

11.3 Notebook interativo

A completar

Versão do notebook de partida com *sliders* para c e precisão, sobrepondo o gráfico calculado, as fronteiras x_1, x_2 e a previsão da fórmula (Plotly).

11.4 Erros comuns

A completar

Catálogo curto: usar `float32` sem perceber (arrays NumPy), somar um valor pequeno a um grande em laço, comparar `==` após somas, e confiar na impressão com poucas casas.

Capítulo 12

Conclusões e perspectivas

12.1 O que se estabeleceu

O comportamento de $(x^n + c) - x^n$ em `binary32` e `binary64` é determinado pela ULP de x^n e pela posição dos bits de c. Sob arredondamento ao mais próximo e as hipóteses H1–H4, f é dada pela fórmula (4.1): exata até $x_1 = 2^{(p+\nu)/n}$, ruidosa durante L binades (o número de bits significativos de c), e nula a partir de $x_2 = 2^{(p+1+\lambda)/n}$. O erro em ULPs vale no máximo $\frac{1}{2}$ e o erro de f é sempre representável.

12.2 O que fica em aberto

- $c < 0$, $c \sim y$ e a cota exata na região $y < c$ (Capítulos 4 e 9).
- A fórmula fechada da frequência do efeito de borda.
- O papel de FMA, de precisão estendida e do cálculo de y.
- Uma leitura completa das obras que não pude abrir, para confirmar o que é conhecido e o que é novo (veja a nota abaixo).

Revisão da literatura: situação

A Seção 1.4 e o Apêndice D registram a revisão feita. Os ingredientes e o mecanismo (Fast2Sum, exatidão de z, erro da soma representável) estão na literatura [11, 12, 14]. Nas fontes lidas não apareceu a forma explícita em $\nu(c)$, λ, L, x_1 e x_2. Faltam a leitura direta do *Handbook* [3], de [2], do artigo original de [1] e da tese [18], além de uma busca nas atas da conferência ARITH.

12.3 Perspectivas

- Formatos de menor precisão (`float16`, `bfloat16`): $x_1 = 2^{p/10}$ é pequeno, e o fenômeno aparece em escalas triviais. Fora do escopo deste texto.
- Análise probabilística do erro de somas longas, usando L e o dente de serra como bloco de construção.
- Versão formalizada da Proposição 4.1 em um assistente de provas (Coq/Flocq, Isabelle).

Apêndice A

Código

Os três arquivos reproduzem todos os números e figuras do texto. O código parte do notebook `numError01` de E. R. L. D. Ribas (MIT).

A.1 `ulp_core.py`: modelo e experimento

```
"""Nucleo do experimento: f(x) = (x^n + c) - x^n em ponto flutuante, ULP e modelo teorico."""
import math
import numpy as np

P = {np.float64: 53, np.float32: 24}   # bits de precisao (com o bit implicito)

def v2(c, dt=np.float64):
    """Expoente do menor bit 1 de c (c = m * 2^v2, m impar)."""
    p = P[dt]
    m, e = np.frexp(dt(c))
    mi = int(float(m) * 2**p)
    tz = (mi & -mi).bit_length() - 1
    return e - p + tz

def floor_log2(c, dt=np.float64):
    return int(np.frexp(dt(c))[1]) - 1

def theory(c, n=10, dt=np.float64):
    """Previsoes: faixa de binades (expoente E de y=x^n) em que f e ruidoso, e limiares em x."""
    p = P[dt]; c = dt(c)
    E1 = p + v2(c, dt)                  # 1a binade inexata (u = 2^(v2+1): empate)
    E2 = p + floor_log2(c, dt)          # ultima binade com f != 0
    return dict(E1=E1, E2=E2, L=E2 - E1 + 1,
                x1=2.0**(E1 / n), x2=2.0**((E2 + 1) / n))

def f_fl(x, c, n=10):
    y = x**n
    return (y + c) - y, y

def predict_binade(c, E, dt=np.float64):
    """Conjunto de valores previstos de f na binade [2^E, 2^(E+1)) de y."""
    p = P[dt]; c = float(dt(c))
    u = 2.0**(E - (p - 1))
    q = c / u
    fl = math.floor(q)
    if q - fl == 0.5:                       # empate: depende da paridade da mantissa de y
```

```
        return {fl * u, (fl + 1) * u}
    return {math.floor(q + 0.5) * u}

def sample_binade(c, E, n=10, dt=np.float64, N=4000, rng=None):
    """Amostra x com y=x^n em binades vizinhas de E; devolve (E_real, f)
        agrupavel."""
    rng = rng or np.random.default_rng(0)
    lo, hi = 2.0**(E / n), 2.0**((E + 1) / n)
    x = (lo + (hi - lo) * rng.random(N)).astype(dt)
    r, y = f_fl(x, dt(c), n)
    Ereal = np.frexp(y)[1].astype(int) - 1
    return Ereal, r.astype(np.float64), y
```

A.2 validate.py: validação da fórmula

```
import numpy as np, math, json, sys
from ulp_core import *
rng = np.random.default_rng(42)

def cases(dt):
    p=P[dt]
    odd=[float(k) for k in range(1,2002,2)]
    dy=[]
    for _ in range(300):
        j=int(rng.integers(1,9)); m=int(rng.integers(1,2**int(rng.integers(1,10))
            )*2+1) if rng.random()<.9 else 1
        dy.append(m/2**j)
    nd=[0.1,0.2,0.3,1/3,0.7,0.6,0.9,math.pi,math.e,2/3]+[float(v) for v in rng.
        uniform(0.01,50,300)]
    return dict(impar=odd, diadico=dy, nao_diadico=nd)

def check(c, n, dt, N=3000):
    th = theory(c,n,dt)
    ok=True; ncheck=0; bad=[]
    cross=0; tot_s=0
    for E in range(max(th['E1']-2, floor_log2(c,dt)+1), th['E2']+3):
        Er, r, y = sample_binade(c,E,n,dt,N,rng)
        for Ek in np.unique(Er):
            if Ek!=E: continue
            cr = (y.astype(np.float64)+float(dt(c))) >= 2.0**(Ek+1)
            sel = (Er==Ek) & ~cr
            cross += int(((Er==Ek)&cr).sum()); tot_s += int((Er==Ek).sum())
            if not sel.any(): continue
            got=set(np.unique(r[sel]).tolist())
            # previsao: exato (=c) fora da zona; modelo dentro e depois
            if Ek < th['E1']: pred={float(dt(c))}
            else: pred=predict_binade(c,Ek,dt)
            ncheck+=sel.sum()
            if not got<=pred or (Ek>=th['E1'] and Ek<=th['E2'] and got!=pred and
                len(pred)==2 and N>1000 and len(got)==1):
                ok=False; bad.append((Ek,sorted(got),sorted(pred)))
    return ok,ncheck,bad,(cross,tot_s)

res={}
CR={'float64':[0,0],'float32':[0,0]}
for dt in (np.float64,np.float32):
    n=10
    for cls,cs in cases(dt).items():
        tot=0; nok=0; badl=[]
        for c in cs:
```

```
            if dt is np.float32 and c>1e6: continue
            ok,nc,bad,cr=check(c,n,dt); CR[dt.__name__][0]+=cr[0]; CR[dt.__name__
                ][1]+=cr[1]
            tot+=1; nok+=ok
            if not ok: badl.append((c,bad[:2]))
        res[(dt.__name__,cls)]=(nok,tot,badl[:3])
        print(dt.__name__,cls,f"{nok}/{tot} c com modelo 100% correto")
        for b in badl[:3]: print("   ruim:",b)

for k,v in CR.items(): print(k,'amostras que cruzam a binade:',v[0],'de',v[1],f'
    ({v[0]/max(v[1],1):.2e})')
```

A.3 `figs.py`: figuras

```
import numpy as np, math
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm, LinearSegmentedColormap
from ulp_core import *

plt.rcParams.update({"font.size":10,"axes.spines.top":False,"axes.spines.right":
    False,
    "axes.grid":True,"grid.alpha":.25,"figure.dpi":100,"savefig.dpi":170,"savefig
        .bbox":"tight"})
BLUE="#0072B2"; ORG="#E69F00"; RED="#D55E00"; GRN="#009E73"; GREY="#666666"
OUT="out/"

# ---------- Fig 1: degraus para c impar ----------
def fig1():
    cs=[3,5,101,173]
    fig,axs=plt.subplots(2,2,figsize=(11,7))
    for ax,c in zip(axs.ravel(),cs):
        th=theory(c)
        x=np.linspace(th['x1']-0.8, th['x2']+1.2, 14000)
        r,y=f_fl(x,np.float64(c))
        ax.axvspan(2**(th['E1']/10),2**((th['E1']+1)/10),color=ORG,alpha=.18,lw
            =0,label="binade de empate (u = 2)")
        ax.axhline(c,color=GREY,ls="--",lw=1,label="valor exato c")
        ax.plot(x,r,".",ms=1.6,color=BLUE,label="f(x) calculado")
        for E in range(th['E1'],th['E2']+2):
            ax.axvline(2**(E/10),color=GREY,lw=.6,ls=":")
        ax.set_title(f"c = {c}   (L = {th['L']} binades ruidosas)",fontsize=10.5)
        ax.set_xlabel("x"); ax.set_ylabel("f(x)")
        ax.set_xlim(x[0],x[-1])
        ax.annotate("f = 0",(th['x2']+.6,0),xytext=(0,10),textcoords="offset
            points",ha="center",color=RED,fontsize=9)
        pass
    h,l=axs[0,0].get_legend_handles_labels()
    fig.legend(h,l,loc="lower center",ncol=3,frameon=False,markerscale=5,
        bbox_to_anchor=(.5,-.02))
    fig.tight_layout(rect=(0,.03,1,1))
    fig.savefig(OUT+"fig1_degraus_impar.png"); plt.close(fig)

# ---------- Fig 2: mapa (x, c) para c impar ----------
def fig2():
    cs=np.arange(1,256,2,dtype=np.float64)
    x=np.linspace(36,82,2300)
    Z=np.empty((len(cs),len(x)))
    for i,c in enumerate(cs):
```

```
        r,_=f_fl(x,c); Z[i]=np.abs(r-c)/c
    Zm=np.ma.masked_where(Z==0,Z)
    cmap=plt.get_cmap("magma_r").copy(); cmap.set_bad("white")
    fig,ax=plt.subplots(figsize=(10.5,5.6))
    dx=x[1]-x[0]
    m=ax.pcolormesh(np.append(x-dx/2,x[-1]+dx/2),np.append(cs-1,cs[-1]+1),Zm,norm
        =LogNorm(1e-3,1),cmap=cmap,shading="flat",rasterized=True)
    cb=fig.colorbar(m,ax=ax,pad=.01); cb.set_label("erro relativo |f(x) ? c| / c
          (branco = exato)")
    ax.axvline(theory(1)['x1'],color=BLUE,lw=1.4)
    ax.text(theory(1)['x1']+.3,252,"x1 = 2^(53/10) ? 39,40",color=BLUE,ha="left",
        va="top",fontsize=9,bbox=dict(fc="white",ec="none",pad=1.5,alpha=.9))
    # x2(c) teorico
    cc=np.arange(1,257); x2=[2**((54+floor_log2(c))/10) for c in cc]
    ax.step(x2,cc-.5,where="post",color=GRN,lw=1.6)
    ax.step(x2,cc-.5,where="post",color=GRN,lw=0)
    xs=[];ys=[]
    for c in range(1,257):
        t=theory(c); xs+= [t['x2'],t['x2']]; ys+=[c-1+0.0,c+0.0]
    ax.plot([],[],color=GRN,lw=1.6,label="x2(c) = 2^((54+?log2c?)/10)  (previsto)
        ")
    ax.legend(loc="lower right",frameon=True,facecolor="white",framealpha=1,
        fontsize=9)
    ax.set_xlabel("x"); ax.set_ylabel("c (inteiro impar)"); ax.set_xlim(36,82);
        ax.set_ylim(0,256)
    ax.grid(False)
    fig.tight_layout(); fig.savefig(OUT+"fig2_mapa_impar.png"); plt.close(fig)

# ---------- Coleta para Fig 3 / Fig 5 ----------
def collect(c,dt=np.float64,n=10,N=2500,rng=None):
    rng=rng or np.random.default_rng(3)
    th=theory(c,n,dt); rows=[]; cd=float(dt(c))
    meas_noisy=[]
    for E in range(max(th['E1']-1,floor_log2(c,dt)+1), th['E2']+2):
        Er,r,y=sample_binade(c,E,n,dt,N,rng)
        sel=(Er==E)&((y.astype(np.float64)+cd)<2.0**(E+1))
        vals=np.unique(r[sel]); u=2.0**(E-(P[dt]-1))
        if len(vals)==0: continue
        noisy = not (len(vals)==1 and vals[0]==cd)
        meas_noisy.append((E,noisy))
        for v in vals: rows.append((c,E,cd/u,(v-cd)/u,u))
    return rows,meas_noisy,th

def fig3():
    rng=np.random.default_rng(11)
    cl={"inteiros impares":[float(k) for k in range(1,400,2)],
        "fracoes diadicas":[(2*k+1)/2**j for j in range(1,7) for k in range
            (0,40,3)],
        "fracoes nao diadicas":[0.1,0.2,0.3,1/3,0.7,2/3,math.pi,math.e
            ,1.1,2.3,5.7,0.45]}
    col={"inteiros impares":BLUE,"fracoes diadicas":GRN,"fracoes nao diadicas":
        RED}
    mk={"inteiros impares":"o","fracoes diadicas":"s","fracoes nao diadicas":"^"}
    fig,ax=plt.subplots(figsize=(8.6,4.6))
    q=np.linspace(0,1,2001); ax.plot(q,np.where(q<.5,-q,1-q),color="k",lw=1.3,
        label="previsao: ? = arred(q) ? q",zorder=1)
    ax.plot([.5,.5],[-.5,.5],color="k",lw=1,ls=":",zorder=1)
    allmax=0; tot=0
    for k,cs in cl.items():
        pts=[]
        for c in cs:
            rows,_,th=collect(c,rng=rng,N=1500)
```

```
            for (_,E,qq,eu,u) in rows:
                if E>=th['E1'] and E<=th['E2']: pts.append((qq%1.0,eu))
        a=np.array(pts); tot+=len(a); allmax=max(allmax,np.max(np.abs(a[:,1])))
        ax.scatter(a[:,0],a[:,1],s=14,marker=mk[k],color=col[k],alpha=.45,label=f"{k} (n = {len(a)})",zorder=2,lw=0)
    ax.set_xlabel("parte fracionaria de q = c / ulp(x10)"); ax.set_ylabel("erro em ULPs:  ? = (f ? c) / ulp(x10)")
    ax.set_ylim(-.6,.6); ax.axhline(.5,color=GREY,lw=.8,ls="--"); ax.axhline(-.5,color=GREY,lw=.8,ls="--")
    ax.legend(frameon=False,fontsize=8.5,loc="upper left",markerscale=1.4)
    fig.tight_layout(); fig.savefig(OUT+"fig3_dente_serra.png"); plt.close(fig)
    print("fig3: pontos",tot,"max|?|",allmax)

# ---------- Fig 4: fracionarios ----------
def fig4():
    cs=[(0.75,"c = 3/4  (diadico, 2 bits)",False),(0.0625*5,"c = 5/16  (diadico, 3 bits)",False),(0.1,"c = 0,1  (nao diadico)",True),(1/3,"c = 1/3  (nao diadico)",True)]
    fig,axs=plt.subplots(2,2,figsize=(11,7))
    for ax,(c,t,nd) in zip(axs.ravel(),cs):
        th=theory(c)
        lo=max(th['x1']*0.6,0.2) if nd else th['x1']-1.2
        hi=th['x2']+(1.5 if nd else 1.0)
        x=np.linspace(lo,hi,40000); r,y=f_fl(x,np.float64(c)); e=(r-c)/c
        ax.axvspan(th['x1'],th['x2'],color=ORG,alpha=.13,lw=0)
        ax.axvline(th['x1'],color=ORG,lw=1.2); ax.axvline(th['x2'],color=RED,lw=1.2)
        if nd:
            pos=e>0; neg=e<0
            ax.plot(x[pos],e[pos],".",ms=2,color=BLUE,label="f > c")
            ax.plot(x[neg],-e[neg],".",ms=2,color=RED,label="f < c  (|erro| mostrado)")
            ax.set_yscale("log"); ax.set_ylim(3e-18,3); ax.set_ylabel("|f ? c| / c")
            ax.axhline(2.0**-53,color=GREY,ls="--",lw=1); ax.text(lo+(hi-lo)*0.02,2.0**-53*1.6,"2?53",ha="left",fontsize=8,color=GREY)
            ax.legend(frameon=False,fontsize=8,loc="upper left",markerscale=4)
        else:
            ax.plot(x,e,".",ms=2,color=BLUE); ax.set_ylabel("(f ? c) / c")
            ax.axhline(0,color=GREY,lw=.8)
        ax.set_title(f"{t}:  x1 = {th['x1']:.2f},  x2 = {th['x2']:.2f},  L = {th['L']}",fontsize=10)
        ax.set_xlabel("x"); ax.set_xlim(lo,hi)
    fig.tight_layout(); fig.savefig(OUT+"fig4_fracionarios.png"); plt.close(fig)

# ---------- Fig 5: faixas ruidosas por valor de c ----------
def fig5():
    rng=np.random.default_rng(5)
    lst=[("2?1 (potencia de 2)",0.5,GRN),("1 (potencia de 2)",1.0,GRN),("26 (potencia de 2)",64.0,GRN),
         ("3",3.0,BLUE),("7",7.0,BLUE),("101",101.0,BLUE),("1001",1001.0,BLUE),("65537",65537.0,BLUE),
         ("3/4",0.75,ORG),("5/16",0.3125,ORG),
         ("0,1",0.1,RED),("1/3",1/3,RED),("?",math.pi,RED)]
    fig,axs=plt.subplots(1,2,figsize=(12.5,5.4))
    for ax,dt,nm in zip(axs,(np.float64,np.float32),("float64 (p = 53)","float32 (p = 24)")):
        for k,(lab,c,colr) in enumerate(lst):
            c=float(dt(c)); th=theory(c,10,dt)
            ax.barh(k,th['x2']-th['x1'],left=th['x1'],height=.62,color=colr,alpha=.55,lw=0)
            rows,mn,_=collect(c,dt,N=1500,rng=rng)
```

```
142             noisy=[E for E,nz in mn if nz]
143             ax.plot([2**(min(noisy)/10),2**((max(noisy)+1)/10)],[k,k],"|",color="
                    k",ms=13,mew=1.6)
144             ax.text(th["x2"]*1.12,k,f"L = {th['L']}",va="center",fontsize=8.5,
                    color=GREY)
145         ax.set_yticks(range(len(lst))); ax.set_yticklabels([l for l,_,_ in lst]);
                ax.invert_yaxis()
146         ax.set_xscale("log",base=2); ax.set_xlim(0.5,160 if dt is np.float64 else
                24)
147         ax.set_xticks([0.5,1,2,4,8,16,32,64,128] if dt is np.float64 else
                [0.5,1,2,4,8,16])
148         ax.get_xaxis().set_major_formatter(matplotlib.ticker.ScalarFormatter())
149         ax.set_xlabel("x"); ax.set_title(nm); ax.grid(axis="y",alpha=0)
150     import matplotlib.patches as mp
151     h=[mp.Patch(color=GRN,alpha=.55,label="c = 2k (caso ja explorado)"),mp.Patch(
            color=BLUE,alpha=.55,label="c inteiro impar"),
152        mp.Patch(color=ORG,alpha=.55,label="c fracao diadica"),mp.Patch(color=RED,
               alpha=.55,label="c nao diadico"),
153        plt.Line2D([],[],marker="|",color="k",ls="",ms=12,mew=1.6,label="limites
               medidos")]
154     fig.legend(handles=h,loc="lower center",ncol=5,frameon=False,bbox_to_anchor
            =(.5,-.02))
155     fig.tight_layout(rect=(0,.04,1,1)); fig.savefig(OUT+"fig5_faixas_ruido.png");
            plt.close(fig)
156
157 if __name__=="__main__":
158     import sys
159     for f in (sys.argv[1:] or ["fig1","fig2","fig3","fig4","fig5"]):
160         globals()[f](); print("ok",f)
```

Apêndice B

Tabelas completas

A completar

- Tabela de x_1, x_2, L para todos os c ímpares de 1 a 255, nos dois formatos.
- Resultado da validação por classe e precisão (número de c testados, amostras, discrepâncias).
- Frequências do efeito de borda por binade.

Apêndice C

Notação

Símbolo	Significado
p	precisão em bits (24 ou 53)
$\mathbb{F}$	conjunto de números de ponto flutuante
fl, RN	arredondamento ao mais próximo, empate para par
E	binade: $y \in [2^E, 2^{E+1})$
$u = \text{ulp}(y)$	2^{E-p+1}
M	significando inteiro de y: $y = Mu$
$c = m2^\nu$	m ímpar, $\nu = \nu(c)$ o expoente do menor bit
λ	$\lfloor \log_2 c \rfloor$
q	c/u
ε	erro em ULPs: $(f - c)/u$
E_1, E_2	primeira e última binade ruidosa: $p + \nu$ e $p + \lambda$
L	$E_2 - E_1 + 1 = \lambda - \nu + 1$
x_1, x_2	$2^{E_1/n}$ e $2^{(E_2+1)/n}$
τ	zeros à direita do significando inteiro de c ($L = p - \tau$)

Apêndice D

Registro da revisão da literatura

Este apêndice registra o que foi *efetivamente lido* em 4 de novembro de 2026, e com que resultado, para que a revisão possa ser retomada e auditada. A leitura foi feita por extração automática de texto de páginas e PDFs; por isso os enunciados devem ser conferidos nas fontes antes de citados em um trabalho final.

D.1 Fontes lidas

Fonte	O que contém sobre o tema	Forma explícita em $\nu(c)$, L?
Boldo, Graillat, Muller [11]	Fast2Sum com arredondamentos arbitrários; Lema 2.5 ($s-a$ exato se $e_a \geq e_b$); Teorema 3.1 (t fiel ao erro)	Não
Jeannerod, Zimmermann [12]	Lema 2 ($a \in \mathrm{ulp}(b)\mathbb{Z} \Rightarrow z = s-a$); Teorema 1 (RN: $x+y = a+b$); limites de erro	Não
[13]	Condições para qualquer arredondamento fiel e para arredondamento ímpar; contraexemplos para z inexato	Não
Martin-Dorel, Melquiond, Muller [14] (*slides*)	Lema 3 (erro de soma em RN é representável); Teorema 4 (Fast2Sum); $z = s-a$ por Sterbenz	Não
Linnainmaa [15]	Usa $L(a)$, $F(a)$, $W(a)$ (largura de um número em dígitos); cita resultados de soma de trabalho anterior	Não (só noções de largura)
Goldberg, cópia da Oracle [1]	Teoremas 1–7 apenas; absorção descrita só com exemplos em dígitos; sem Teorema 11 (Sterbenz) nem Kahan formal	Não
Notas sobre *swamping* (Arnold)	"Cerca de $m-p$ dígitos se perdem", sem fórmula	Não

D.2 Fontes que não consegui ler

- *Handbook of Floating-Point Arithmetic* [3]: apenas trechos indiretos (os *slides* de [14], que seguem a Seção 4.3). Texto completo não acessível.
- Higham [2]: o capítulo de somas não pôde ser lido além da abertura.
- Goldberg [1]: a cópia aberta lida era um excerto (Teoremas 1 a 7); o apêndice "Details" não foi alcançado.

- Tese de Boldo [18]: a página do HAL bloqueou o acesso; encontrei apenas a ficha bibliográfica. Do trabalho da autora sobre Veltkamp/Dekker [17] só vi a página da galeria Toccata, sem o texto do artigo.
- Linnainmaa, BIT 1974 (citado por [15]) e Rump–Ogita–Oishi [16]: não abertos.

D.3 Busca ainda por fazer

- Ler as Seções 2.2, 4.3 e 6 do *Handbook* (ULP, Fast2Sum, 2Sum, erro de soma) e o Capítulo 4 do Higham.
- Conferir o apêndice de Goldberg (Teoremas 7 a 14) no artigo original.
- Verificar a tese de Boldo (capítulos sobre Dekker/Veltkamp e sobre a exatidão de somas).
- Percorrer as atas ARITH (2015–2026) em busca de “Fast2Sum”, “absorption”, “error-free”.
- Buscar material didático e de *blogs* que descreva “a perda dos bits baixos de c quando x cresce”.

Referências

[1] D. Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.

[2] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*. 2. ed. SIAM, 2002.

[3] J.-M. Muller et al. *Handbook of Floating-Point Arithmetic*. 2. ed. Birkhäuser, 2018.

[4] IEEE. *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019.

[5] P. H. Sterbenz. *Floating-Point Computation*. Prentice-Hall, 1974.

[6] T. J. Dekker. A floating-point technique for extending the available precision. *Numerische Mathematik*, 18:224–242, 1971.

[7] D. E. Knuth. *The Art of Computer Programming, vol. 2: Seminumerical Algorithms*. 3. ed. Addison-Wesley, 1997.

[8] W. Kahan. Further remarks on reducing truncation errors. *Communications of the ACM*, 8(1):40, 1965.

[9] J.-M. Muller. On the definition of ulp(x). Relatório de pesquisa RR-5504, INRIA, 2005.

[10] E. R. L. D. Ribas. Notebook `numError01`. Licença MIT.

[11] S. Boldo, S. Graillat, J.-M. Muller. On the robustness of the 2Sum and Fast2Sum algorithms. *ACM Transactions on Mathematical Software*, 44(1), 2017.

[12] C.-P. Jeannerod, P. Zimmermann. FastTwoSum revisited. *Proc. 32nd IEEE Symposium on Computer Arithmetic (ARITH)*, 2025.

[13] *Odd but error-free FastTwoSum: more general conditions for FastTwoSum as an error-free transformation for faithful rounding modes*. arXiv:2601.17198, 2026. (Autores a conferir.)

[14] É. Martin-Dorel, G. Melquiond, J.-M. Muller. Some issues related to double roundings. *BIT Numerical Mathematics*, 2013 (*slides*, 2012, foram os lidos). (Conferir.)

[15] S. Linnainmaa. Software for doubled-precision floating-point computations. *ACM Transactions on Mathematical Software*, 7(3):272–283, 1981. (Conferir.)

[16] S. M. Rump, T. Ogita, S. Oishi. Accurate floating-point summation, Part I: Faithful rounding. *SIAM J. Sci. Comput.*, 31(1), 2008. (Conferir.)

[17] S. Boldo. Pitfalls of a full floating-point proof: example on the formal proof of the Veltkamp/Dekker algorithms. *IJCAR 2006*, LNCS 4130, pp. 52–66.

[18] S. Boldo. *Preuves formelles en arithmétiques à virgule flottante*. Tese de doutorado, École Normale Supérieure de Lyon, 2004.